

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation. (BL4, BL5)

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

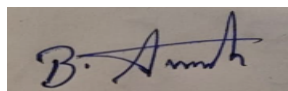
CASE STUDY

Code of Conduct:

I Avinash B (192511193)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A handwritten signature in blue ink, appearing to read 'B. Avinash', is written on a light-colored rectangular background.

Case Study 1: Weak Password Hashing in Web Application

Scenario: A web application stores user passwords using unsalted SHA-1 hashes. After a breach, attackers quickly crack multiple passwords using precomputed tables.

Analysis: how the lack of salting and use of SHA-1 contributed to the vulnerability

- SHA-1 is a general-purpose, deliberately fast cryptographic hash — it was designed for speed, not to resist password guessing. On modern GPUs, billions of SHA-1 hashes can be computed per second, so an attacker can try enormous numbers of candidate passwords very quickly once the hash database is leaked.
- Without a per-user salt, identical passwords across different users produce identical hashes, and — critically — an attacker can use a rainbow table: a huge precomputed lookup table mapping common passwords directly to their hash values. Reversing a hash then becomes a near-instant table lookup rather than a fresh computation, which is exactly why the breach allowed attackers to crack multiple passwords 'quickly.'
- Combined, these two weaknesses meant the stored hashes offered almost no real protection: the algorithm was too fast to slow down brute-force attempts, and the missing salt meant precomputed tables could be reused across the entire user base instead of requiring a fresh attack per account.

Recommendation: a secure password hashing strategy, and justification

Passwords should be stored using a slow, memory-hard, adaptive hashing algorithm designed specifically for password storage — Argon2id (preferred, winner of the Password Hashing Competition), or bcrypt/scrypt as strong alternatives — combined with a unique, randomly generated salt per user and a tunable cost/work factor.

- A random salt per user eliminates the rainbow-table attack entirely, since a precomputed table would now need to be regenerated per salt, which is computationally infeasible at scale.
- Argon2id/bcrypt/scrypt are deliberately slow and, for Argon2id/scrypt, memory-hard, meaning they resist acceleration on GPUs and custom ASIC hardware far better than a simple fast hash like SHA-1, directly countering the brute-force speed advantage attackers had in this breach.
- The work factor can be increased over time as hardware gets faster, keeping the scheme future-proof without requiring a fundamental algorithm change.

Case Study 2: Digital Signature Compromise in Financial System

Scenario: A financial institution uses digital signatures for transaction approval. An attacker gains access to a user's private key and performs unauthorized transactions.

Evaluation: impact on authentication and non-repudiation

Authentication is broken: since the private key is the sole proof of identity in a digital-signature scheme, any signature produced with the stolen key is indistinguishable from a signature the legitimate user actually made. The system can no longer reliably confirm who really authorized a given transaction.

Non-repudiation is undermined: non-repudiation is supposed to mean the legitimate user cannot later deny having signed a transaction they did sign. Once the key is compromised, this guarantee is turned against the victim — the user may now be wrongly held responsible for fraudulent transactions the attacker signed in their name, and there is no cryptographic way to distinguish the attacker's signatures from the user's own, unless independent evidence (e.g., trusted timestamps predating the compromise) exists.

More broadly, every signature ever produced under that key becomes suspect once the compromise is discovered, and trust in the institution's signature-based approval process is damaged until the key is revoked and the incident is contained.

Mitigation strategies to secure private keys and prevent recurrence

- Store private keys in Hardware Security Modules (HSMs) or secure hardware tokens/smart cards, so the key material never exists in an extractable software form that malware or an attacker with system access can copy.
- Require multi-signature / threshold-signature approval for high-value transactions (e.g., 2-of-3 authorized signers), so a single compromised key can never authorize a transaction alone.
- Enforce short-lived certificates and regular key rotation, limiting how long a compromised key remains useful to an attacker even if the compromise goes undetected for a while.
- Maintain rapid revocation capability (CRL/OCSP) so a confirmed compromised key can be immediately invalidated system-wide, and use trusted timestamping of signatures to help establish which signatures pre-date a known compromise.
- Add anomaly/behavioral detection on transaction patterns (unusual amounts, destinations, or timing) as a secondary defense layer that can flag misuse even before the key compromise itself is confirmed.

Case Study 3: Kerberos Authentication Failure in Distributed System

Scenario: A distributed system using Kerberos experiences frequent authentication failures due to time synchronization issues between servers.

Analysis: how time synchronization impacts Kerberos functionality

Kerberos tickets (the initial Ticket-Granting Ticket and subsequent service tickets) are timestamped and carry a limited validity lifetime specifically to prevent replay attacks — an attacker capturing a ticket should not be able to reuse it indefinitely. To enforce this, the protocol allows only a small configurable clock-skew tolerance (5 minutes by default) between the client, the Key Distribution Center (KDC), and the application servers involved.

If a server's or client's clock drifts beyond this tolerance, perfectly legitimate tickets appear to the receiving server as already expired or as issued in the future, and are rejected — producing authentication failures that have nothing to do with an actual attack. This is exactly the symptom described: frequent, unexplained failures caused purely by clock drift across the distributed system rather than any credential problem.

Suggested solutions for reliable authentication

- Deploy Network Time Protocol (NTP) synchronization across every client, server, and the KDC against a reliable, authoritative time source, with periodic automatic re-synchronization rather than relying on manual clock-setting.
- Where possible, use NTS (Network Time Security) or another authenticated time-sync mechanism, so an attacker cannot spoof NTP responses to deliberately desynchronize a target's clock and trigger denial-of-service or manipulate the skew window.
- Monitor clock drift continuously with alerting if any host exceeds a safe threshold, so drift is corrected before it causes authentication failures rather than being discovered only after users are locked out.
- Set the Kerberos clock-skew tolerance to a reasonable value (the standard 5-minute default is a good balance) — widening it excessively to 'fix' the failures would reopen a larger window for replay attacks, trading security for availability.

Case Study 4: Certificate Authority Breach in PKI System

Scenario: A Certificate Authority (CA) is compromised, leading to the issuance of fake X.509 certificates for trusted domains.

Evaluation: impact on system trust and secure communication

Public Key Infrastructure (PKI) trust is transitive: any client that trusts a CA's root will automatically trust every certificate that CA (or its compromised issuance process) signs. A fake certificate issued for a legitimate domain (e.g., a bank's website) will therefore be accepted by browsers and clients as fully valid, letting an attacker impersonate that domain, intercept 'secure' TLS traffic via an undetectable man-in-the-middle attack, and run indistinguishable phishing sites — all while the connection still shows as secure/HTTPS to the victim.

Because trust in PKI is anchored entirely in the CA's integrity, a single compromised CA can damage confidence in every certificate it has ever issued, and — if the CA is broadly trusted — can force browser vendors to distrust it entirely, as happened historically with the DigiNotar breach in 2011, which affected the whole web ecosystem's trust in that CA.

Mechanisms to detect and mitigate CA compromise

- Certificate Transparency (CT) logs: require every issued certificate to be published in public, append-only, cryptographically verifiable logs, so domain owners and monitoring services can watch for certificates fraudulently issued for their domain and detect a breach quickly.
- CAA (Certificate Authority Authorization) DNS records: let a domain owner explicitly declare which CAs are permitted to issue certificates for their domain, so a compromised or rogue CA issuing outside that authorization can be flagged or refused.
- Rapid revocation via CRLs and OCSP (ideally with OCSP stapling) so fraudulently issued certificates can be invalidated system-wide as soon as they are discovered.
- Strong CA operational security: HSM-protected root keys, multi-party signing ceremonies, and regular independent audits (WebTrust/ETSI) to reduce the likelihood of a breach occurring in the first place.
- As a last resort, browser and OS vendors can remove the compromised CA from trusted root stores entirely, cutting off its ability to vouch for any certificate going forward.

Case Study 5: Integrity Verification in Cloud Data Storage

Scenario: A cloud service provider uses hashing to verify file integrity, but users report data tampering incidents. Investigation reveals improper hash validation practices.

Analysis: how incorrect hashing implementation leads to integrity failures

- Unprotected reference hash: if the trusted hash value is stored alongside the data in the same location an attacker can already modify, the attacker can simply update the stored hash to match the tampered file — the hash is not cryptographically bound to anything the attacker cannot also alter, so it provides no real protection against a capable attacker.
- Weak hash algorithm: using a broken or collision-prone hash (e.g., MD5) allows an attacker to craft tampered data that still produces the original hash value (a collision), defeating the integrity check even without touching the stored hash at all.
- Incomplete or inconsistent verification: only checking the hash once at upload time (and not on retrieval or periodically thereafter), using a non-constant-time or buggy comparison routine, or silently ignoring

verification failures instead of rejecting the data, all mean tampering that occurs after upload can go completely undetected in practice, even though hashing is nominally 'in place.'

Recommendation: a robust integrity verification mechanism, and justification

- Use a strong, currently uncompromised hash function (SHA-256 or SHA-3) and verify it on every access/retrieval — not only once at upload — so tampering is caught whenever the data is used, not just missed after the initial check.
- Store the trusted reference hash separately from the data, in an access-controlled location the same attacker who could modify the file cannot also modify (or better, sign the hash, or compute an HMAC over it with a key unavailable to whoever has write access to the stored files), so the hash itself cannot be silently updated to match tampered data.
- For large datasets, use a Merkle tree structure: this allows efficient, verifiable proofs that a specific block within a large file or dataset is untampered without rehashing everything, and lets a third-party auditor verify integrity without needing full access to the data — an approach widely used in cloud storage and blockchain systems.

This combination is effective because it addresses each root cause identified above: a strong hash removes the collision risk, a protected/signed reference value prevents an attacker from tampering with the hash alongside the data, and continuous (not one-time) verification ensures tampering is detected whenever it occurs rather than only if someone happens to check immediately after upload.